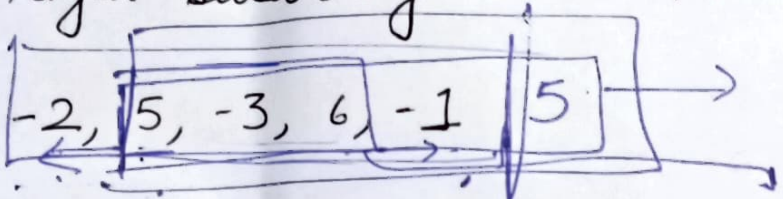
PROBLEM **Maximum Sum Contiguous Subarray**

→ Given an array A[] of n numbers, find the maximum sum found in any contiguous subarray

→ A zero length subarray has maximum 0

→ e.g 

maximum sum be 5-3+6 = 8

e.g A[] = 10, 15, -3, -4, -2, -1, 8, 5

maximum sube 28 (all numbers added together)

SOLUTION 1

BRUTE FORCE

0	1	2	3
-1	3	2	-4

Size 1: {-1} {3} {2} {-4}

Size 2: {-1,3} {3,2} {2,-4}

Size 3: {-1,3,2} {3,2,-4}

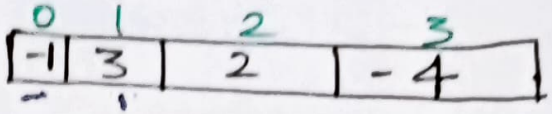
Size 4: {-1,3,2,-4}

$\frac{n(n+1)}{2}$

$O(n^2)$

Pseudocode

Max = -∞
for (i = 0 to n do
    sum = 0
    for j = i to n do
        sum = sum + arr[j]
        if (Max < sum)
            Max = sum
        endif
    endfor
endfor
end.

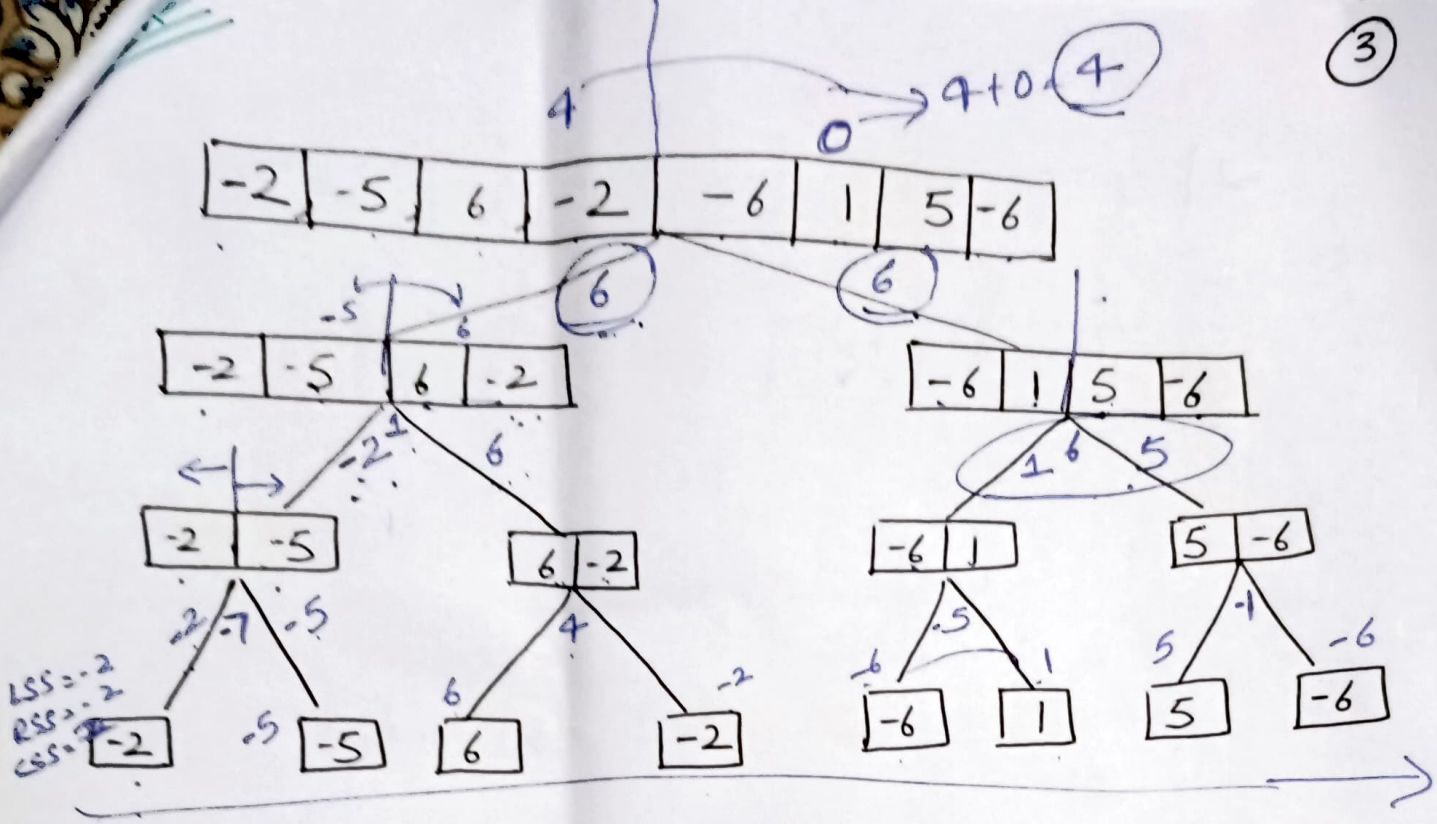


i = 0

j = 0 [-1]
j = 1 [-1, 3]
j = 2 [-1, 3, 2]
j = 3 [-1, 3, 2, -4]

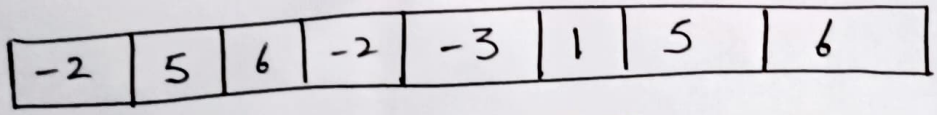
i = 1

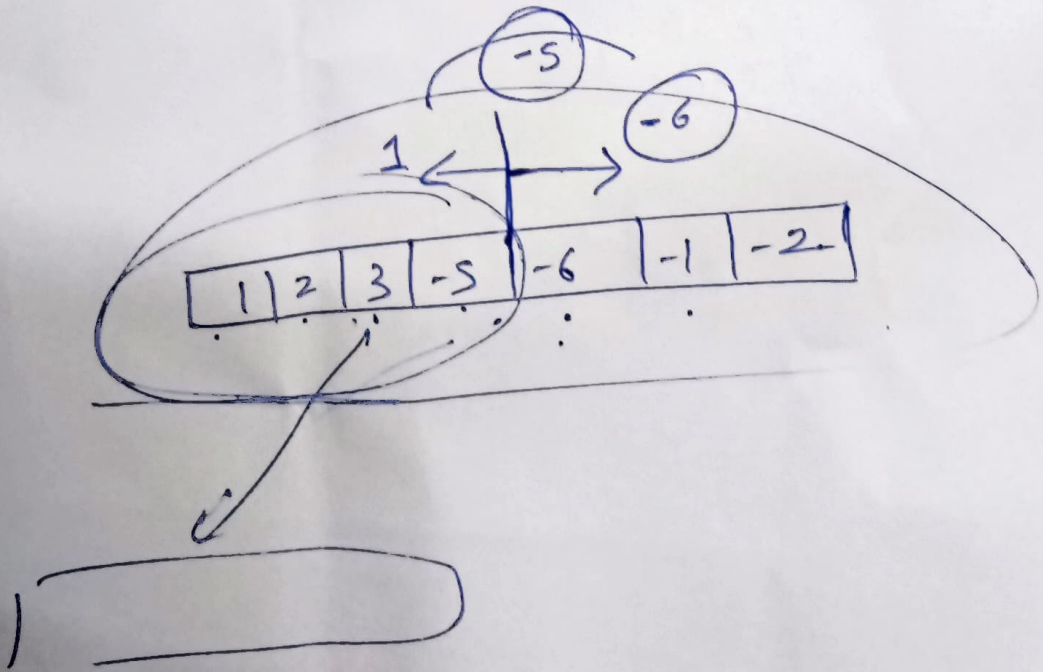
j = 1 | 3
j = 2 | 3 2
j = 3 | 3 2 -4

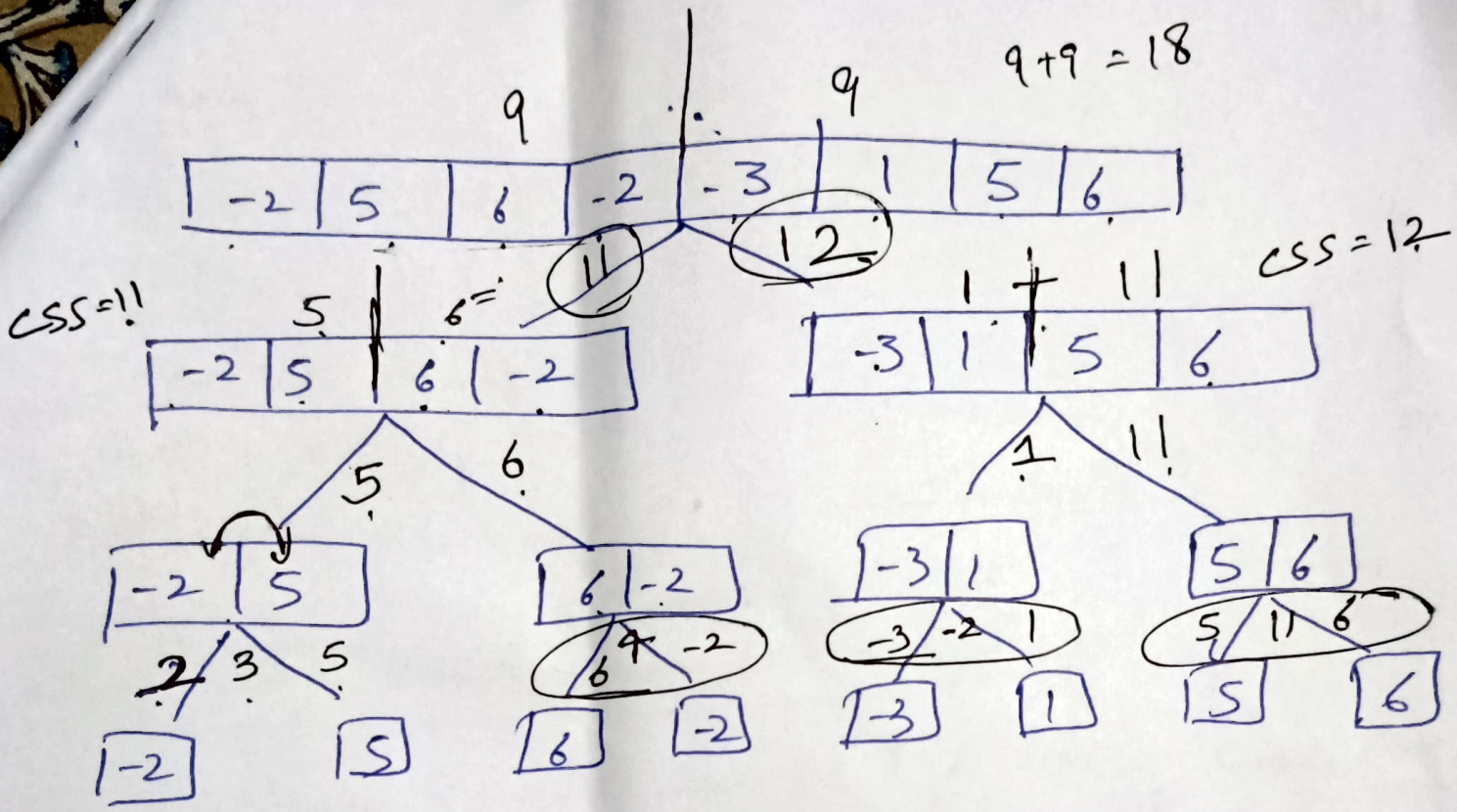


→ At each step we will find the max(LSS, RSS, CSS)

- LSS: Left sum subarray
- RSS: Right sum subarray
- CSS: Cross sum subarray






9+9 = 18
9
9
-2 5 6 -2 -3 1 5 6
11
12
CSS=11
5
6
-2 5 6 -2
CSS=12
1
11
-3 1 5 6
5
6
1
11
-2 5
6 -2
-3 1
5 6
2
3
5
6 4 -2
-3 -2 1
5 11 6
-2
5
6
-2
-3
1
5
6

MaxSum (A, l, h) ⟶ T(n)

```
if (l==h)
  return A[l];
m = l+(h-l)/2;
leftsum = MaxSum(A, l, m)        ⟶ T(n/2)
rightsum = MaxSum(A, m+1, h)     ⟶ T(n/2)
crosssum = Maxcrosssum(A, l, m, h);   n
```

```
if (leftsum >= rightsum && leftsum >= crosssum)
    return (leftsum)
else if (rightsum >= leftsum && rightsum >= crosssum)
    return (rightsum)
else
    return (crosssum)
```

Recurrence Relation:

$$T(n) = \begin{cases} 1 & n=1 \\ 2T(n/2) + n & n>1 \end{cases}$$

Peak Index in Mountain Array:

PROBLEM

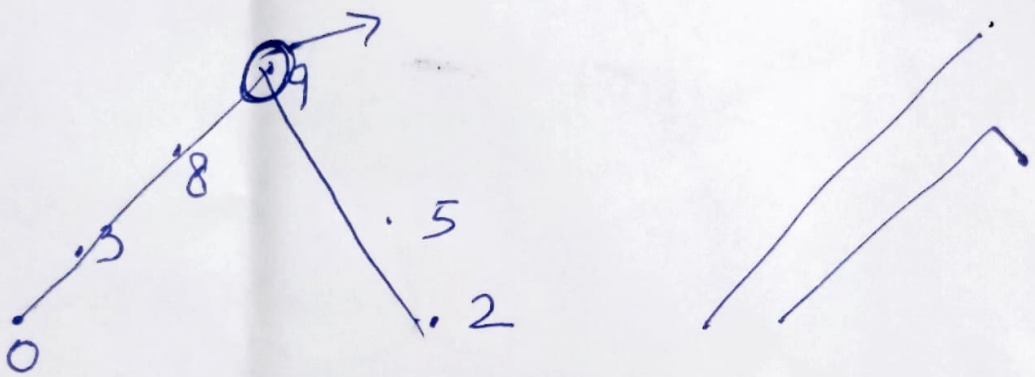
→ Given a Mountain Array, we have to find the index of peak value.

e.g

0	1	2	3	4	5
0	3	8	9	5	2

what is mountain Array?

Plot



PROPERTIES

① $a[P-i] < a[P] > a[P+i]$.

② Minimum of 3 elements are reqd.

on 1

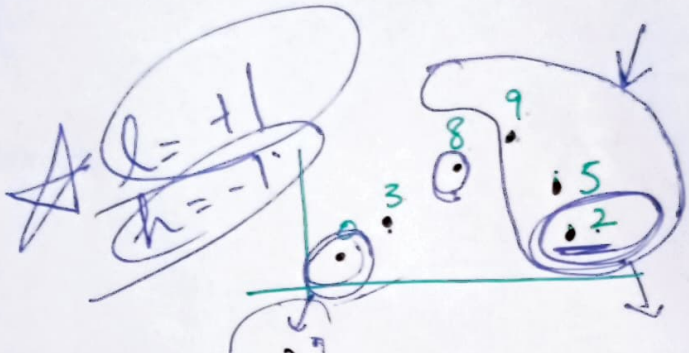
① Linear Search

Solution 2

② Binary search

Iterative

```
while (l <= h){
    mid = l + (h - l)/2
    if (A[mid-1] < A[mid] > A[mid+1])
        return mid
    if (A[mid-1] < A[mid])
        l = mid+1
    else
        h = mid-1
```



Recursive

Correctness

$$T(n) = \begin{cases} T(n/2) + 1 & n > 1 \\ 1 & n = 1 \end{cases}$$

You have an array of n elements. Suppose you implement quicksort by always choosing the central element of the array as the pivot. Then the tightest upper bound for the worst case performance is

(A) $O(n^2)$

(b) $O(n \log n)$

(c) $\Theta(n \log n)$

(d) $O(n^3)$

Q2 A binary search tree T contains n distinct elements. What is the time complexity of picking an element in T that is smaller than the maximum element in T?

(A) $\Theta(1)$

(b) $\Theta(n \log n)$

(c) $\Theta(\log n)$

(d) $\Theta(n)$

Q3 Consider the following recurrence relation.

$$T(n) = \begin{cases} \sqrt{n}\, T(\sqrt{n}) + n & \text{for } n \geq 1 \\ 1 & \text{for } n = 1 \end{cases}$$

Q4

$$T(n) = \begin{cases} T(n/2) + T(2n/5) + 7n & n > 0 \\ 1 & n = 0 \end{cases}$$

$T(n) = \sqrt{n}\, T(\sqrt{n}) + n \Rightarrow \log n$

$n = 2^m$

$T(n) = 2^{m/2} T(2^{m/2}) + 2^m$

$S(m) = T(2^m) \Rightarrow$

$S(m) = 2^{m/2} S(m/2) + 2^m$

$2^m f(m) = S(m)$

~~$f(m) = f(m/2)$~~

$2^m f(m) = 2^{m/2}\left(2^{m/2} f(m/2)\right) + 2^m$

$2^m f(m) = 2^m f(m/2) + 2^m$

$f(m) = f(m/2) + 1$

$f(m) \quad O(\log m)$

$S(m) \quad 2^m \log m$

$T(n) = n \log\log n.$